



GUÍA PRÁCTICA · INTRODUCCIÓN A LA INGENIERÍA DE SOFTWARE

GIT & GITHUB

Control de versiones y trabajo colaborativo en el desarrollo de software

INTEGRANTES DEL EQUIPO

Carlos Quintana

Edú Plúas

Estick Viñan

Jonatan Acosta

¿Qué es el control de versiones y por qué es importante?

DEFINICIÓN

Un sistema de control de versiones (VCS) es una herramienta que registra, organiza y gestiona cada cambio realizado en los archivos de un proyecto a lo largo del tiempo. Guarda quién cambió qué, cuándo y por qué, formando una línea de tiempo completa del código.

EJEMPLO COTIDIANO

Es como el "Ctrl+Z" de un editor de texto... pero aplicado a todo un proyecto y a todo un equipo a la vez. Cada "commit" es un punto de guardado al que siempre puedes volver, incluso si otra persona hizo cambios después.

¿POR QUÉ ES IMPORTANTE?



Evita perder trabajo

Cada cambio queda respaldado; nada se pierde por un error o un borrado accidental.



Colaboración sin pisarse

Varias personas editan el mismo proyecto sin sobrescribir el trabajo ajeno.



Historial completo

Se puede consultar quién cambió qué y cuándo, en cualquier momento.



Volver atrás cuando haga falta

Si algo falla, el proyecto regresa a una versión anterior estable.

Sistemas centralizados vs. distribuidos



Centralizado (ej. SVN)

Existe un único servidor central que guarda el proyecto completo. Cada desarrollador se conecta a ese servidor para trabajar; sin conexión al servidor, no se puede consultar el historial ni guardar cambios.

- ✦ Un solo punto de fallo: si el servidor cae, nadie trabaja
- ✦ Requiere conexión constante al servidor central
- ✦ El historial completo vive en un solo lugar



Distribuido (ej. Git)

Cada colaborador tiene una copia completa del repositorio en su propia máquina: todo el código y todo el historial. No depende de un único servidor para funcionar.

- ✦ Se puede trabajar sin conexión a internet
- ✦ Redundancia: cada copia es un respaldo del proyecto
- ✦ Operaciones locales (commits, historial) son casi instantáneas



Sin internet



Redundancia



Más rápido

Comandos básicos de Git



```
$ git init
$ git add archivo.txt
$ git commit -m "mensaje"
$ git status
$ git log
$ git clone <url>
$ git push origin main
$ git pull origin main
$ git branch nueva-rama
$ git merge nueva-rama
```

init

Crea un repositorio Git nuevo en la carpeta actual.

add

Prepara los archivos modificados para el próximo commit.

commit

Guarda los cambios preparados con un mensaje descriptivo.

status

Muestra qué archivos cambiaron y su estado actual.

log

Lista el historial de commits del repositorio.

clone

Copia un repositorio remoto completo a la máquina local.

push / pull

Sube cambios locales al remoto / descarga cambios del remoto.

branch / merge

Crea líneas de trabajo paralelas y luego las combina.

Flujo de trabajo básico con Git y GitHub



EL CICLO COMPLETO

Se crea o clona un repositorio, se trabaja localmente haciendo cambios y guardándolos con commits, y luego se suben a GitHub con push. Desde ahí, otros colaboradores pueden clonar o hacer pull del proyecto, proponer cambios mediante pull requests y trabajar en ramas (branches) paralelas sin afectar el código principal hasta que el equipo aprueba la fusión (merge).

Demo: crear un repositorio en GitHub

1

Iniciar sesión en GitHub

Entrar a github.com y acceder a la cuenta del equipo o proyecto.

2

Clic en “New repository”

Desde el botón “+” en la esquina superior, elegir “New repository”.

3

Nombrar el repositorio

Elegir un nombre claro y descriptivo para el proyecto.

4

Elegir visibilidad

Definir si el repositorio será público (cualquiera lo ve) o privado (solo el equipo).

5

Agregar un README

Marcar la opción para generar un archivo README.md inicial con la descripción del proyecto.

6

Crear el repositorio

Confirmar con “Create repository”: el proyecto queda listo en la nube.



Create a new repository

Repository name

mi-proyecto-git

Description (optional)

Proyecto de práctica de Git y GitHub

Visibility

• Public

Private



Add a README file

Create repository

Demo: clonar el repositorio en la máquina local

```
$ git clone https://github.com/equipo/mi-  
proyecto.git  
Cloning into 'mi-proyecto'...  
remote: Enumerating objects, done.
```

Este comando descarga una copia completa del repositorio remoto — incluyendo todo el historial de commits— a una nueva carpeta en la computadora local.

HTTPS vs. SSH para clonar

HTTPS

Pide usuario/token; fácil de configurar.

SSH

Usa llaves; no vuelve a pedir contraseña.



Demo: commits incrementales

Tres cambios pequeños al mismo archivo, cada uno guardado con su propio commit.



Commit 1

Agregar línea

Se añade una nueva función al archivo

```
$ git add app.py  
$ git commit -m "Agrega función saludo"
```



Commit 2

Agregar otra línea

Se suma una segunda funcionalidad

```
$ git add app.py  
$ git commit -m "Agrega validación de  
datos"
```



Commit 3

Corregir un error

Se corrige un bug detectado en pruebas

```
$ git add app.py  
$ git commit -m "Corrige error en  
validación"
```

```
$ git log --oneline
```

```
c3f1a9e  Corrige error en validación  
a7e42b1  Agrega validación de datos  
9d0c8f4  Agrega función saludo
```

El historial muestra los 3 commits en orden, del más reciente al más antiguo.

Demo: conflicto de merge y cómo resolverlo



Dos ramas, misma línea

Dos personas (o dos ramas) editan la misma línea de un archivo de forma distinta. Al intentar unir (merge) ambas ramas, Git no puede decidir automáticamente cuál versión conservar.

```
$ git merge rama-alterna
CONFLICT (content): Merge conflict in app.py
Automatic merge failed; fix conflicts and
commit.
```



Git detiene el merge y marca el archivo en conflicto para que el equipo decida.

app.py

```
<<<<<< HEAD
mensaje = "Hola equipo"
=====
mensaje = "Hola a todos"
>>>>>> rama-alterna
```

PASOS PARA RESOLVERLO

- 1 Abrir el archivo y localizar las marcas <<<<<<, ===== y >>>>>>
- 2 Elegir qué parte del código conservar (una, otra, o una combinación)
- 3 Eliminar las marcas de conflicto una vez decidido el contenido final
- 4 Guardar y ejecutar git add + git commit para cerrar el conflicto



Ideas clave para recordar



El control de versiones protege el trabajo del equipo y todo su historial.



Git es distribuido: cada copia local es completa e independiente.



GitHub añade colaboración en la nube: ramas, pull requests y revisión de código.



Los conflictos de merge son normales; se resuelven revisando el código con calma.

¡Gracias! Preguntas y demo abierta.